

AGENTLOOPGATE: Evidence-Governed Evolution and Fail-Closed Selection for Agent Harnesses

JiaoyangLi

jiaoyanglily@gmail.com

<https://github.com/jiaoyangli-shadow7day/AgentLoopGate>

Draft – August 2026

Abstract

Language-model agents can revise prompts, tools, routing, and control flow, yet a proposed harness change is not evidence that it should ship. Aggregate scores can conceal capability exchange, retries and missing traces can corrupt denominators, and optimizers can evaluate their own proposals. We present AGENTLOOPGATE, a runtime-independent release-governance layer that preserves native trace, persistence, and telemetry while binding evidence, cost, and lineage to frozen objectives, splits, evaluators, pricing, and gates. External updaters may propose candidates; promotion remains human-controlled. P0–R14 incidents are reported as system-formation rather than pooled effectiveness evidence. In a frozen Banking validation with DeepSeek Harness and an external Agentic Harness Engineering updater, we executed all 125 authorized pre-release positions. The baseline and three candidates scored 6/15, 6/15, 5/15, and 6/15 on identical Selection tasks. Every candidate gained task 017 but regressed baseline successes 062 and 095; all paired 95% bootstrap intervals included zero. AGENTLOOPGATE therefore returned HOLD for every candidate and made no Release calls. Exact known model cost was USD 3.908664788, with no unknown cost. Headless fixtures confirmed coexistence with DeepSeek Harness persistence and OpenTelemetry. No positive self-evolution effect is claimed; the result shows that fail-closed, task-paired governance makes abstention auditable.

1 Introduction

An agent harness turns a language model into a system that can observe state, call tools, preserve memory, recover sessions, and interact with users. Recent work improves such systems through linguistic reflection [10, 7], learned prompt and pipeline parameters [4], function-level optimization [13], or automated search over agent programs [3]. These methods broaden what can be changed. They do not, by themselves, establish what evidence is sufficient to release a change.

This distinction matters in stateful, tool-using domains. A candidate may exchange one capability for another while preserving its aggregate score. A retry may make a run appear successful while hiding the failed attempt and its cost. A missing trace or evaluator incident may silently shrink the denominator. The same data can leak from diagnosis into selection. Finally, observability data may help debug a system without being complete enough to serve as release evidence.

We introduce AGENTLOOPGATE, an independent evidence-governance and evolution decision layer. It is not another runtime, trace backend, or autonomous deployment system. The host-native execution log remains the source of what happened. AGENTLOOPGATE adds verifiable references, immutable evaluation identities, bounded updater inputs, independent baseline-paired selection, and a lexicographic gate that returns SHIP-RECOMMENDED, HOLD, or REJECT. A human remains the only promotion authority.

This paper makes four contributions:

1. a runtime-independent evidence model that binds host traces to normalized runs, candidates, snapshots, costs, and decisions without replacing native persistence or telemetry;
2. a fail-closed evolution protocol that separates diagnosis, Update-Check, baseline-paired Selection, and conditional Release, with exact attempt and cost accounting, derived through an auditable P0–R14 sequence of failures and versioned controls;
3. a native DeepSeek Harness plugin that consumes public session events, reconciles flush snapshots, preserves JSONL/SQLite and OpenTelemetry, and isolates governance failures from the agent turn; and
4. a fully sealed Banking study showing that two aggregate ties concealed two-for-two capability exchanges, for which abstention was the correct system outcome.

The empirical claim is deliberately narrow: AGENTLOOPGATE detected unsupported evolution and stopped it before Release. We do not claim that the external updater improved the agent, that the result generalizes beyond Banking, or that fixture latency represents production load.

2 Problem and Design Goals

Let A_0 be a frozen baseline harness and let an external updater produce candidates $C_1, \dots, C_m$ from a bounded failure bundle. For a task t in a frozen Selection population, $y_t(A) \in \{0, 1\}$ denotes stable success under harness A . The release problem is not merely $\arg \max_A \sum_t y_t(A)$. A candidate should be eligible only when the evidence population is complete and its gains do not trade away protected baseline behavior.

Goals. The system should (i) accept proposals from multiple updater families without trusting updaters; (ii) keep diagnosis, selection, and release data disjoint; (iii) make every conclusion traceable to immutable task, run, candidate, snapshot, and pricing identities; (iv) treat retries, timeouts, unknown cost, and infrastructure invalidity as first-class facts; (v) preserve the host runtime’s native operation; and (vi) make abstraction a successful terminal result.

Non-goals. AGENTLOOPGATE does not train model weights, certify agent safety, replace host logs, provide a trace user interface, or deploy a selected candidate. Banking is a reference validation pack rather than a product boundary.

Trust boundary. The trusted kernel contains schemas, canonical serialization, content hashes, split enforcement, the evaluator interface, candidate registry, attempt and cost ledgers, and the gate. Updater workspaces and mutable harness assets are untrusted. Raw model text, user simulation, and external telemetry cannot declare their own success.

3 System Design

3.1 Evidence planes

The evidence path has four planes. H0 is the host-native fact source: a DeepSeek Harness append-only session log or a benchmark raw result. L0 stores a *SourceTraceRef* and a redacted evidence receipt. L1 stores normalized run, tool, state, latency, usage, and cost facts. L2 stores evaluation artifacts: failure bundles, candidate and snapshot manifests, batch summaries, statistics, lineage, and decisions. Each object uses canonical JSON or YAML and a SHA-256 digest. Formal conclusions must traverse the chain back to H0.

Telemetry is intentionally not elevated to evaluation truth. OpenTelemetry is a vendor-neutral framework for traces, metrics, and logs [8]; an exporter can be useful yet absent, sampled, or reconfigured. AGENTLOOPGATE therefore coexists with OTel but bases integrity on the host session event stream and authenticated receipts.

3.2 Frozen identities and immutable lifecycle

A formal experiment freezes the Objective, six isolated task pools, evaluator, pricing, runtime pins, protocol, study plan, source revision, and baseline snapshot. Every paid operation first writes a durable **STARTED** event and then exactly one terminal event. Resume verifies existing bytes and continues only absent work; it cannot silently re-run a paid completed position.

Cost has two distinct questions: whether valid-run comparison cost is exact, and whether the whole observed attempt surface is complete. Missing cost is **partial** or **unavailable**, never zero. Infrastructure-invalid runs are reported outside the performance denominator and block a batch whose registered coverage is incomplete.

3.3 Bounded external updater

The updater receives only a failure bundle derived from Update-Source plus a whitelist of mutable parent-snapshot assets. It cannot read Selection or Release tasks, gold answers, evaluator code, prior private decisions, or the trusted kernel. Outputs are validated for path scope, schema, semantic distinctness, runtime capability binding, and lineage before registration. Updater-native order is retained as evidence but is not a release decision.

3.4 Baseline-paired selection and conditional release

Update-Check cheaply rejects malformed or clearly harmful candidates. The survivors and A_0 then run on the identical Selection tasks. For candidate C , define the paired difference $d_t = y_t(C) - y_t(A_0)$, the gain set $G_C = \{t : d_t = 1\}$, and regression set $R_C = \{t : d_t = -1\}$. The R15 correctness gate required

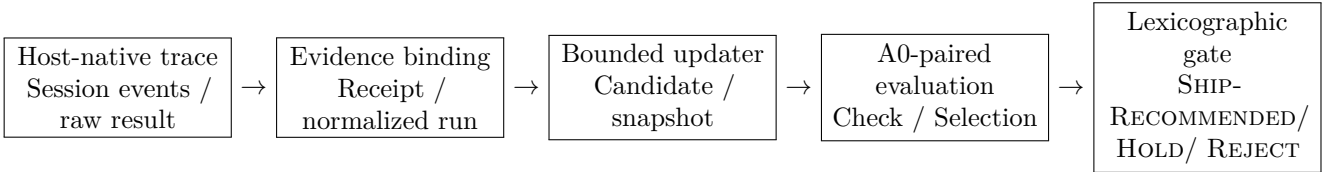
$$\sum_t d_t > 0, \quad R_C = \emptyset, \quad (1)$$

plus complete evidence, zero critical violations, no retry or timeout increase, p95 latency ratio at most 1.2, and whole-attempt cost ratio at most 1.2. A scalar-score tie is not a gain.

If no candidate satisfies every prerequisite, Selection emits HOLD and forbids Release calls. If a candidate is selected, Release-ID, Release-OD, and Replay still require a separately bound execution authorization. Even a final SHIP-RECOMMENDED is a recommendation rather than deployment.

4 DeepSeek Harness Integration

DeepSeek Harness is a plugin-composed agent runtime whose durable session log records the model context, tool calls, and lifecycle events [1]. AGENTLOOPGATE integrates as an out-of-tree Cordis bundle against a pinned



Frozen Objective, Split, Evaluator, Pricing, Protocol, and Study identities bind every transition. Release is created only after Selection returns a candidate and separate authority is present; promotion is never automatic.

Figure 1: AGENTLOOPGATE separates runtime facts, governance evidence, candidate generation, and release decisions. The host trace remains the execution source of truth; content-addressed governance artifacts refer back to it.

developer-preview revision. The Python core remains usable without the plugin.

The observer subscribes to public session lifecycle and event interfaces. A live path provides low-latency receipts; each flush reconciles against the authoritative `session.events` snapshot to recover construction, resume, or transient-buffer gaps. Sequence holes mark evidence incomplete. The plugin does not register a competing telemetry backend, query private SQLite tables, or rewrite native JSONL/SQLite. A bridge failure is isolated from the agent turn but blocks affected evidence from formal decisions. Unload removes tools, listeners, and the bridge child process.

This arrangement makes native trace and governance evidence complementary. Developers retain their session persistence and OTEL exporter; AGENTLOOPGATE adds content-addressed provenance and decision semantics.

5 System Formation and Failure Experience: P0–R14

AGENTLOOPGATE was not designed in one pass. Its current contracts emerged through a sequence of frozen experiments, failed preflights, partial batches, and incident reviews. The repository’s experiment identities progress from `EXP_BANKING_P0` directly to `EXP_BANKING_R2`; no registered `EXP_BANKING_R1` identity exists. We preserve that naming gap instead of inventing a run. P0–R14 also cannot be pooled into an effectiveness curve: protocols, evidence schemas, baselines, and sometimes evaluator inputs changed between identities. Their legitimate role is process evidence—what failed, whether the system stopped, what evidence survived, and which control became mandatory in the successor.

Execution integrity before optimization. P0–R6 showed that an optimizer is premature when a retry can escape its budget, a discarded call can disappear from cost, or a reply representation can turn explicit intent into Infrastructure Invalid. The sequence therefore made attempt topology, timeout ownership, reply normalization, and direct lineage part of the trusted kernel. Importantly, restoring a parseable or complete turn did not rewrite business outcomes: repaired positions could remain valid task failures.

Decision integrity before release. R7–R10 moved the fault boundary downstream. Complete execution was still not enough when dynamic price lookup could create false exact zeros, a summary could read the wrong cost plane, a broken gold fixture could contaminate the failure bundle, or Selection could force a winner without a paired baseline. These incidents produced the frozen-price authority, evaluator versioning, candidate semantic and capability checks, baseline-paired Selection, and first-class HOLD semantics used in R15.

Fail-fast behavior without evidence deletion. R11–R14 tested recovery and stopping behavior. The system repeatedly preserved partial evidence and refused to fill a missing denominator. R13 quantified a real efficiency failure inside a serial batch; R14 then demonstrated that the new position-level stop blocked the remaining queue, while exposing the next control-plane defect before Selection. This is the practical meaning of fail-closed evolution: failure is retained as evidence, not hidden to keep the experiment moving.

Taken together, P0–R14 support a systems claim, not a positive evolution claim. They show that the architecture was shaped by observable failures and that each discovered failure was converted into an executable successor control. R15 is the first complete confirmatory application of the resulting baseline-bound pre-release policy.

6 Banking R15 Methodology

6.1 Reference environment

The study uses a knowledge-intensive Banking environment derived from the τ family of tool-agent-user benchmarks. Such benchmarks evaluate stateful conversations, policy following, API use, and final environment state rather than static text alone [11]. Stateful tool-use evaluation is also emphasized by ToolSandbox [6]. We use Banking to exercise the complete evidence path, not to define the domain of AGENTLOOPGATE.

The execution model and user simulator were DeepSeek endpoints under frozen runtime arguments. The external updater was `agentic-harness-engineering` 0.1.0 at commit `8b2a55d97590363fe50c3cc6b5e833b020a4bb4c`. The benchmark runtime, protocol, study, source tree, pricing,

Round	Terminal evidence	Failure exposed	Control carried forward
P0–R2	P0 preserved as superseded HOLD; R2 A4 retained a failed protocol diagnosis	Frozen one-retry intent differed from the invoked runtime default; stale acceptance, source packaging, ignore rules, and Linux async behavior also failed clean-room checks	Content-address the execution protocol; supersede rather than rewrite; require source-artifact and clean-checkout Linux verification before paid evidence
R3	22/25 valid Update-Source positions; HOLD	Outer resume reapplied the per-process retry budget, and discarded attempts lost User Simulator cost	Global per-position attempt cap; append-only task-attempt and independent Agent/User usage ledgers; unknown cost is never zero
R4	24/25 valid; HOLD; exact whole-attempt model cost USD 0.6917880208	A recurrent, uniquely mappable reply wrapper remained unrecognized	Narrow allow-list normalization only when tool identity and arguments are explicit; ambiguous syntax still fails closed
R5–R6	R5: 23/25 valid; R6: 24/25 valid; both HOLD. R6 reconciled exact whole-attempt cost USD 0.7698343648	Reasoning-only empty finals, equal inner and outer timeouts, additional explicit reply shapes, and indirect failed-call lineage	One separately ledgered same-session final-only repair; ordered timeouts; direct task/attempt/session joins; no inference from hidden reasoning
R7–R8	R7 completed Source, created three candidates, and completed A0 Check; the first candidate Check failed partial. R8 made zero paid calls	Dynamic pricing produced false exact-zero User cost for nonzero tokens; later audit found that summaries and gates still read raw rather than direct-ledger cost	Frozen-price recomputation with a positive-token invariant; direct cost must propagate through summaries, curves, and final gates end to end
R9	25/25 Source valid; immutable HOLD; exact whole-attempt cost USD 0.7015112488	Task 053’s golden fixture omitted a required user action, making a correct trajectory look wrong	Deterministic no-model cause isolation; versioned evaluator overlay; block candidate generation under contaminated evidence
R10	95 formal positions; paid HOLD; exact model cost through C2 USD 3.0651904832	Selection omitted A_0 , forced a winner, ignored several whole-attempt operational facts, admitted duplicate candidates, and allowed an unbound tool route	Same-pool baseline pairing; first-class abstention; whole-attempt retry/timeout/p95/cost inputs; semantic deduplication and runtime capability binding
R11–R12	R11: 24/25 Source valid. R12: 34/35 executed positions valid; both HOLD. R12 observed Provider-cost lower bound USD 1.1970712488	Failed-attempt sealing lacked direct lineage; a User Simulator empty message and updater use of process-global /tmp broke successor execution	Verify-only recovery for existing bytes; bounded final-only User repair; attempt-local updater runtime and sandbox doctor; every repair requires a new identity
R13	24/25 Source valid; HOLD. After permanent invalidity, 12 queued positions still made 588 calls costing a known USD 0.4686327800	Cross-stage gating worked, but the serial batch lacked position-level fail-fast	Stop before the next position after retry exhaustion while retaining the failed position, unknown cost scope, native trace, and incomplete denominator
R14	46/125 positions executed; HOLD; exact cost USD 1.4876297096. Fail-fast prevented 79 authorized positions from starting	Candidate precheck inspected baseline bytes and patch tokens rather than validating materialized prospective bytes; the only complete candidate Check regressed a baseline success	Apply each patch in an isolated tree and validate final bytes before registration; seal a first-class partial HOLD; resume is verification-only

Table 1: System formation through P0–R14. Counts and costs are observed historical evidence, not comparable estimates of candidate effectiveness. Every successor retained the predecessor’s raw evidence and used a new frozen identity when execution semantics changed.

and A_0 were content-addressed before formal execution.

6.2 Registered pre-release surface

R15 executed 25 Update-Source positions, 40 Update-Check positions (A_0 plus three candidates, 10 each), and 60 Selection positions (A_0 plus three candidates, 15 each): 125/125 authorized positions across nine complete batches. All positions were valid and no Infrastructure Invalid run occurred. The experiment contained no Release authorization.

The external updater emitted three semantically distinct candidates in native order, labeled C1–C3. Selection used stable task success as its primary endpoint. Latency, critical violations, retries, timeouts, and whole-attempt cost were gate inputs rather than post hoc diagnostics.

6.3 Uncertainty and multiplicity

For each candidate, we bootstrap the 15 paired task differences d_t with 10,000 deterministic resamples and re-

port the nearest-rank 95% interval for $\bar{d}$. The bootstrap seed was preregistered and each comparison derives an effective content-bound seed. The task, not the trial or model call, is the statistical unit [2]. Intervals are descriptive for the frozen Selection population: three proposals from one updater are dependent, and we make neither a multiplicity-adjusted superiority claim nor a cross-domain inference.

6.4 Research questions

We ask whether independent governance (RQ1) detects task-level capability exchange missed by aggregate scores; whether the system abstains before Release when no candidate clears the frozen gate (RQ2); whether evidence and cost remain complete through retries and failures (RQ3); and whether the DeepSeek Harness plugin coexists with native persistence and OTel in headless fixtures (RQ4).

7 Results

7.1 Aggregate ties concealed common regressions

C1 and C3 tied A_0 at 6/15, while C2 scored 5/15. The ties were not stable equivalence. Every candidate gained task 017 and lost the same baseline successes 062 and 095; C2 also lost 056. C1 gained an additional task 096, whereas C2 and C3 gained 090. Thus two aggregate ties represented two gains exchanged for two protected successes. All paired intervals included zero.

Operational evidence independently disqualified every candidate. All p95 latencies exceeded the 1.2 ratio; C1 added one timeout. Whole-attempt cost ratios remained within 1.2 but were higher than A_0 . Because Equation 1 is a prerequisite rather than a weighted score, lower cost could not compensate for regression.

7.2 The system correctly abstained

The final reason was `no_candidate_passed_baseline_bound_selection_policy`. AGENTLOOPGATE held C1–C3, set the governed candidate to null, emitted HOLD, and made zero model calls after Selection. No Release-ID, Release-ODD, Replay, promotion, or deployment artifact exists. This is a normal, successful terminal state: the system refused to manufacture a winner from a candidate ladder with unsupported evidence.

RQ1 and RQ2 are therefore answered positively for governance behavior, not for candidate quality. Independent paired selection changed an updater-native ranking into abstention, and the conditional Release boundary prevented additional paid evaluation after a failed prerequisite.

7.3 Evidence and cost reconciled exactly

All nine batches carried exact accounting. The call ledger contains 4,378 terminal model calls, 10,343,844 input tokens, 4,648,289 output tokens, 222,019,328 cache-read tokens, zero unresolved calls, and zero provider-level retries. Task-level retry and timeout facts were separately retained in batch evidence. Formal batches cost USD 3.8830976464; external updater calls cost USD 0.0255671416; exact known model cost was USD 3.9086647880. Unknown model-cost scope was empty. Local compute and GitHub Actions monetary cost were explicitly unmetered or unavailable, not reported as zero.

The formal process wall time was 52,591.59 seconds (14.61 hours). This duration includes the deliberately serial, low-concurrency execution path and should not be read as a minimal achievable runtime. It demonstrates that path, duration, retries, calls, tokens, and cost can be retained as evidence rather than reconstructed after the fact.

7.4 Plugin coexistence passed at fixture scope

A registered no-model ablation ran 30 paired iterations for each of JSONL and SQLite persistence with 100 logical session events per iteration. In every iteration, native persistence survived, the logical session-event hash was equivalent, observer coverage was complete, and OTel coexistence remained true. JSONL overhead had p50 –0.308 ms and p95 16.424 ms; SQLite overhead had p50 0.244 ms and p95 0.542 ms. Negative samples reflect local timing noise. These measurements validate lifecycle coexistence in a headless fixture, not production throughput or tail-latency service objectives.

A separate synthetic integrity control illustrates gate value: with evaluation integrity disabled, the fixture would return SHIP-RECOMMENDED; the production gate returns HOLD on `evaluation_integrity`. Because this is a synthetic control, it is evidence of software behavior rather than a causal estimate from R15.

8 Discussion

What evolved? The external updater produced three real, executable harness proposals, and all three changed the task-success surface. That is evidence of candidate generation, not positive evolution. R15 instead identifies a directional hypothesis: future updaters should treat the stable baseline-success set as protected constraints and propose sparse, failure-cluster-specific changes. Testing that hypothesis requires a new frozen identity and untouched confirmation tasks; R15 cannot validate a mechanism designed after seeing its Selection failures.

Why not optimize a weighted score? A weighted objective permits cost or latency to compensate for cor-

Variant	Stable	Gains	Regress.	Paired $\bar{d}$ [95% CI]	p95 ratio	Cost ratio	Disposition
A_0	6/15	—	—	—	1.000	1.000	baseline
C1	6/15	017, 096	062, 095	0.000 [−.267, .267]	1.882	1.121	held
C2	5/15	017, 090	056, 062, 095	−.067 [−.333, .200]	1.670	1.108	held
C3	6/15	017, 090	062, 095	0.000 [−.267, .267]	1.652	1.127	held

Table 2: Frozen Selection results on the identical 15-task population. Cost ratio uses whole observed attempt cost; p95 ratio is relative to A_0 . Every candidate violated strict gain, zero regression, and p95 non-inferiority.



Figure 2: Left: candidate gains and regressions relative to A_0 on the paired Selection population. Right: the fail-closed gate outcome. These are Selection-only figures; Release was not run. Both panels are deterministic views bound to Selection digest `b5e800...00466`.

rectness regression. In high-stakes action domains, this can make policy-important capabilities disappear behind an unchanged average. Lexicographic prerequisites express the stronger release contract: evidence first, then protected correctness, then operational non-inferiority. Different applications can freeze different gates, but changing them after results would require a new study.

Trace versus telemetry. Observability and evaluation evidence overlap but are not identical. Native session logs answer what the runtime recorded. Otel answers what configured instrumentation exported. AGENTLOOPGATE keeps the native log authoritative and adds receipts that are purpose-built for decision completeness. This avoids forcing developers to abandon existing persistence or exporters.

Negative results as systems evidence. Agent-evolution research naturally favors examples where a method improves an aggregate metric. A governance system should also be evaluated when no proposal deserves release. R15 exercises the more adversarial branch: plausible, executable candidates change behavior but fail protected constraints. Correct abstention, exact accounting, and proof that no Release tail started form the primary result.

9 Related Work

Language-agent improvement. ReAct interleaves reasoning and acting [12]; Reflexion stores linguistic feedback across trials [10]; and Self-Refine iteratively

critiques outputs [7]. DSPy compiles declarative LM pipelines against metrics [4]. AgentOptimizer treats functions as learnable agent parameters [13], while Automated Design of Agentic Systems searches over agent programs [3]. AGENTLOOPGATE is complementary: it does not propose a superior optimization algorithm, but constrains inputs, authenticates outcomes, independently selects proposals, and governs whether evaluation may advance.

Agent evaluation and tool safety. AgentBench evaluates agents across interactive environments [5]; τ -bench focuses on tool-agent-user interaction and reliability [11]; ToolSandbox adds stateful on-policy tool execution [6]; and ToolEmu uses an emulated sandbox to identify risky tool behavior [9]. AGENTLOOPGATE uses such benchmark adapters as evidence producers but adds lifecycle, split, lineage, cost, and release semantics across experiments.

Reproducibility and governance. Classical bootstrap methods quantify uncertainty from empirical populations [2]. AGENTLOOPGATE binds the exact population, seed, batch digests, and comparison artifact so an interval cannot drift independently of the underlying runs. Its contribution is operational: statistical output is one prerequisite within a content-addressed decision path, not a substitute for evidence integrity.

10 Limitations and Ethics

R15 covers one Banking environment, one updater backend, one model/runtime family, three dependent candi-

dates, and a 15-task Selection population. Bootstrap intervals are correspondingly wide and descriptive. No candidate reached Release, so there is no Release-ID, OOD, Replay, or cross-domain effectiveness evidence. The plugin overhead study is a local synthetic fixture on arm64 macOS, not a production load test. User simulation and provider variance may affect outcomes. A formal human-factors study of gate review is also absent.

The study does not expose raw customer or session content. Public artifacts are sanitized aggregates; raw traces, prompts, model responses, credentials, and hidden benchmark content remain private. Secret and direct-PII scans are part of clean-room verification. The system never auto-promotes. These controls reduce accidental release risk but do not prove safety against a malicious host, compromised evaluator, or all forms of prompt injection.

11 Reproducibility and Artifact Availability

The source tree contains the Python governance core, DeepSeek Harness bundle, no-key demo, frozen configuration identities, unit and integration tests, and a sanitized 18-file R15 v1.1 result package. The package independently verifies without raw traces. Its Manifest digest is `sha256:79e9a8dd31009f969cdd79021bbcc857c827dc1`

`d5a6808a28fd05937e4f364c8`; the private Outcome digest it binds is `sha256:827ed2a5b3337832c49cd255f17568fad3a58fe201860311d66decdb83ba0bdc3`.

The unified clean-room command runs Python tests and demo, builds an sdist and wheel, type-checks and tests the plugin, executes headless conformance, packs the bundle, verifies both R15 packages, and scans the intended public tree for secrets and direct PII. The repository is prepared at the URL in the title block; visibility and arXiv submission remain separate owner-authorized actions at the time of this draft.

12 Conclusion

AGENTLOOPGATE separates the ability to generate an agent-harness change from the authority to release it. Its evidence contracts preserve host-native traces, its immutable lifecycle makes retries and cost auditable, and its A0-paired gate makes capability exchange visible. In Banking R15, every updater proposal gained behavior, every proposal also regressed protected successes, and no proposal demonstrated a strict stable improvement. The system’s correct result was therefore HOLD with zero Release calls. This negative candidate result is a positive governance result: self-evolution becomes a claim that must be earned by evidence rather than inferred from an optimizer’s output.

References

- [1] DeepSeek AI. Deepseek harness: Everything is a plugin. <https://github.com/deepseek-ai/deepseek-harness>, 2026. Developer preview; accessed 2026-08-25.
- [2] Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979.
- [3] Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. *arXiv preprint arXiv:2408.08435*, 2024.
- [4] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. DSPy: Compiling declarative language model calls into self-improving pipelines. *arXiv preprint arXiv:2310.03714*, 2023.
- [5] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xunyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. Agentbench: Evaluating LLMs as agents. *International Conference on Learning Representations*, 2024.
- [6] Jiarui Lu, Thomas Holleis, Yizhe Zhang, Bernhard Aumayer, Feng Nan, Felix Bai, Shuang Ma, Shen Ma, Mengyu Li, Guoli Yin, Zirui Wang, and Ruoming Pang. Toolsandbox: A stateful, conversational, interactive evaluation benchmark for LLM tool use capabilities. *arXiv preprint arXiv:2408.04682*, 2024.
- [7] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems*, 36, 2023.
- [8] OpenTelemetry Authors. Opentelemetry specification. <https://opentelemetry.io/docs/specs/otel/>, 2026. Accessed 2026-08-25.
- [9] Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. Identifying the risks of LM agents with an LM-emulated sandbox. *International Conference on Learning Representations*, 2024.
- [10] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36, 2023.
- [11] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ -bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024.
- [12] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. *International Conference on Learning Representations*, 2023.
- [13] Shaokun Zhang, Jieyu Zhang, Jiale Liu, Linxin Song, Chi Wang, Ranjay Krishna, and Qingyun Wu. Offline training of language model agents with functions as learnable weights. *arXiv preprint arXiv:2402.11359*, 2024.

A Evidence Binding

Table 3 lists the principal content identities used by this paper. Full values are included so that the manuscript can be checked against the sanitized package without private execution data.

Artifact	Digest or identity
Experiment	EXP_BANKING_R15
Protocol	sha256:b40198176f9de2b39c9433131455bc0af07eee54650327622677cd9085b84a5f
Study	sha256:bd5988f4f89068fa6347728f6b88ee4b8d8d1f644931ec110944e06b74e25043
Execution source	tree:sha256:0e3d0794bd21c5f20ff78512144e50da37b16a9784fa948a13055f6c847b4c8e
Selection	sha256:b5e800793f700e1d096a325961b3f40bac8cd93d12dddbdc111d48fa600466
Statistics	sha256:8bc2da84cefc7ed100947401a6277d9fab49109ce29d065f91b064a3a2f5b8ba
Outcome	sha256:827ed2a5b3337832c49cd255f17568fad3a58fe201860311d66dec83ba0bdc3
Public package	sha256:79e9a8dd31009f969cdd79021bbcc857c827dc1d5a6808a28fd05937e4f364c8

Table 3: Content-addressed identities for the R15 result.

B Formation-Evidence Audit Map

Table 4 binds the P0–R14 narrative to tracked historical records. These artifacts remain historical context: they are not silently promoted into the R15 decision population.

Rounds	Principal repository evidence
P0–R4	configs/formal_experiment.yaml; docs/adr/0002-version-formal-execution-protocol-for-banking-r2.md; SPEC.md Sections 16.4–16.6; R3/R4 execution and reply calibrations under artifacts/research/
R5–R8	artifacts/research/banking_r5/execution_diagnosis.json; banking_r6/execution_diagnosis.json; banking_r7/execution_diagnosis.json; banking_r8/cost_incident_diagnosis.json
R9–R10	artifacts/research/banking_r9/experiment_hold.json; banking_r9/eval_incident_task_053.json; banking_r10/selection_design_correction.json
R11–R12	docs/research/banking-r11-preregistration.md; artifacts/research/banking_r12/formal_execution_seal.json; docs/research/banking-r12-successor-plan.md
R13–R14	artifacts/research/banking_r13/formal_execution_seal.json; banking_r13/fail_fast_incident.json; banking_r14/formal_execution_seal.json; docs/research/banking-r14-terminal.md

Table 4: Audit sources for the system-formation narrative. Artifact paths after the first full prefix remain relative to artifacts/research.

C Candidate-Level Operational Values

The baseline whole-attempt cost was USD 0.4520428360. C1, C2, and C3 cost USD 0.5066522160, USD 0.5009778704, and USD 0.5095613824. Their p50/p95 latencies in milliseconds were 264,917/1,838,752; 273,388/1,631,601; and 314,154/1,614,647, compared with A_0 at 265,903/977,157. C1 and A_0 each recorded one task retry; only C1 recorded a timeout. No candidate had a critical violation or an Infrastructure Invalid run.

D Claim-to-Artifact Audit Map

The repository includes a non-model verifier that checks the manuscript’s author identity, P0–R14 formation claims, R15 decision, scores, paired changes, intervals, costs, usage, ablations, digests, citations, claim limits, and figure presence against tracked artifacts:

```
uv run python -m scripts.verify_arxiv_paper --project .
```

Paper claim	Sanitized R15 evidence source
Selection scores, paired gain/regression sets, gate findings, and governed null candidate	<code>selection.json</code> ; <code>reports/selection_hold.json</code>
Paired bootstrap intervals, statistical unit, seeds, and multiplicity scope	<code>statistics.json</code>
Exact batch and whole-experiment model cost	<code>cost_summary.json</code>
Calls, tokens, provider retries, failures, and operation duration	<code>failure_accounting.json</code>
Zero post-Selection calls, zero Release batches, and terminal HOLD	<code>selection_hold_outcome.json</code>
Plugin persistence/OTel coexistence and synthetic integrity control	<code>ablations/plugin_coexistence_overhead.json</code> ; <code>ablations/integrity_gate.json</code>
Frozen source, protocol, study, and package identities	<code>reproduction.json</code> ; <code>manifest.json</code>

Table 5: Claim-level map within the 18-file sanitized R15 v1.1 package. Paths are relative to `artifacts/research/banking_r15/release_v2`.

Rebuilding or checking this manuscript makes no provider model calls and does not re-run R15. Formal experiment replay would require a new frozen identity and remains outside the paper build.